

Hardware :: False Sharing & Coherency Contention

(Hardware-Level Cache Line Bouncing)

1. The Anatomy of False Sharing

False sharing occurs when two threads on different cores update different variables that happen to share the same **64-byte cache line**.

Even though the variables are logically independent, the CPU's **MESI (Modified/Exclusive/Shared/Invalid) coherence protocol** operates at the *cache line* level, not the *byte* level.

The "Ping-Pong" Effect:

1. **Core A** modifies `metrics[0].count`. It requests **Exclusive (E)** ownership of the cache line from the L1/L2 interconnect.
2. **Core B** modifies `metrics[1].count`. The CPU controller sees that Core A just grabbed the line, so it forces Core A to write back to L3 and invalidates Core A's L1 copy.
3. **Core A** wants to update `metrics[0].sum` in the next cycle. It now has to fetch the line back from L3.

The result? The cache line bounces between Core A and Core B like a ping-pong ball. The CPU essentially serializes execution, and performance drops to the speed of the L3-to-L1 interconnect latency (or worse, the system bus).

2. Why our `Telemetry` class is Immune

We have implemented **Spatial Partitioning**, the standard engineering solution to this problem.

```
struct alignas(64) ThreadStorage {  
    AtomicSlot metrics[kMetricCount];  
};
```

By adding `alignas(64)` to the `ThreadStorage` struct, we ensure that every `ThreadStorage` instance begins at a memory address that is a multiple of 64.

- **The Result:** Because `sizeof(AtomicSlot) * kMetricCount` is likely a multiple of 64 (or if not, the compiler adds padding to reach the next alignment), our array `ThreadStorage`

`slots_[MaxThreads]` becomes a perfectly partitioned layout.

- **The Benefit: Thread 0** writes from `0x...0000` to `0x...003F`. **Thread 1** writes from `0x...0040` to `0x...007F`. They **cannot physically touch the same cache line**.

3. The "Gotcha" (Padding the Struct)

One thing to watch for: If `sizeof(ThreadStorage)` is not *exactly* a multiple of 64, the compiler will pack them tightly, and we might accidentally spill a struct across a cache-line boundary.

Engineering Rule of Thumb: If we want to be paranoid-level safe, we can add an explicit pad to the end of the struct:

```
struct alignas(64) ThreadStorage {
    AtomicSlot metrics[kMetricCount];
    // Explicit padding to ensure the array stays perfectly aligned if
    // sizeof(AtomicSlot) * kMetricCount is not a multiple of 64.
    char padding[64 - (sizeof(AtomicSlot) * kMetricCount % 64)];
};
```

4. The Hardware Reality

When we run this on our Linux/WSL setup, the **L1 Cache Controller** acts as our bodyguard.

When Core 0 performs: `lock add [address_0x00], 1`

The L1 controller marks that line as **Modified**. If Core 1 attempts to perform: `lock add [address_0x40], 1`

The hardware checks the cache directory. It sees `0x40` is a **different cache line** than `0x00`. The coherence controller simply says: *"Core 1 has its own line, Core 0 has its own line. No need to synchronize. Proceed at full speed."*

The "System Engineering" Takeaway

- **Without** `alignas(64)`: We would have unpredictable performance spikes where the kernel would randomly slow down by 10x-50x depending on how the compiler arranged those variables in memory.
- **With** `alignas(64)`: We have achieved **Deterministic Performance**. We have removed the hardware dependency on the compiler's memory layout decisions.

We've basically optimized our telemetry to behave like **Register-local storage** for each core. That is as fast as physical memory can possibly get.